Distributed Message Queue 2
Getting & Monitoring Job Status 3
Job Sink Event Aggregation 4
Hybrid Recommendation System 5
Ad Click Aggregation 6
Ticket Booking System 7
Video and Streaming Leaderboard 8
Payment System 9
Digital Wallet 10
Stock Exchange 11
The Learning Continuum 12

24

Distributed Email Service

In this chapter we design a large-scale email service, such as Gmail, Outlook, or Yahoo Mail. The growth of the Internet has led to an explosion in the volume of email. In 1993, the total email traffic was 10 billion active users and Outlook had over 400 million users worldwide [1].

Figure 1: Popular email providers

Step 1 - Understand the Problem and Establish Design Scope

Email has evolved, and most of our requirements in contemporary email scale. A distributed email service is a complex system with many features. There is no way we can design a real-world system in 45 minutes. So before jumping into the design, we definitely want to ask clarifying questions to narrow down the scope.

Candidate: How many people use the email service?
Interviewer: Assume 1 billion users.

Candidate: What are the important features?
Interviewer: Assume the following features are important:

- Authentication.
- Email delivery.
- Fetch all emails.
- Organize emails by sent/received dates.
- Search emails by subject, sender, and body.
- Anti-spam and anti-virus.

Interviewer: That is good. We don't need to worry about authentication. Let's focus on the other features you mentioned.

Candidate: How do users connect with mail servers?
Interviewer: Traditionally, users connect with mail servers using proprietary protocols. Now most popular email protocols are standard protocols. These protocols are legacy to some extent, yet still very popular. For this interview, let's assume HTTP is used for client and server communication.

Candidate: Any scalability requirements?
Interviewer: Yes.

Non-functional requirements

Next, let's go over the non-functional requirements.

Reliability: We should not lose email data.

Availability: Email is a critical service, so we want to minimize downtime. Availability requirement should be at least four nines.

Scalability: As the number of users grows, the system should be able to handle the increasing number of users and emails. The performance of the system should not degrade when it is overloaded.

Flexibility and extensibility: A distributed system allows us to add new features or improve performance easily by adding new components. Traditional protocols such as POP and IMAP do not have extensibility (more on this in high-level design). Extensibility is not a hard requirement, but it is a nice to have property and capability requirement.

Back-of-the-envelope estimation

Let's do back-of-the-envelope calculations to determine the scale and to discover challenges our solution will need to address. By design, emails are storage heavy applications.

- 1 billion users
- Each person sends/receives an average number of emails per day to 50 (50 for sending emails + 50 * 10 = 550 for receiving emails)
- Average email size is 20 kilobytes (Assume 10 kilobytes on average for text and 10 kilobytes on average for attachments) 30% Metadata storage is necessary for each email, including attachment files.
- Average storage for a single email for 30 years. Storage requirement for maintaining metadata is 1 year * billion users * 30 emails * 30 * 365 days * 10 KB = 10 PB
- Assume 20% of emails contain an attachment and the average attachment size is 500 KB.
- Storage for attachments is 1 year * billion users * 30 emails * 20% * 500 KB = 1000 PB
- Per second transaction (QPS) for the system is calculated, it's clear we need deal with a lot of data. So, it's likely that we need a distributed database solution.

Step 2 - Propose High-Level Design and Get Buy-In

In this section, we first discuss some basic email server protocols and how email servers used to work back in the day. Then we look at the high-level design of distributed email services. The content is structured as follows:

- Email Protocolsp 3D
- Traditional mail servers
- Distributed mail servers

Email knowledge 101

There are various email protocols that are used to send and receive email. Historically, most mail servers use email protocols such as SMTP, IMAP, and POP3.

Email protocols

SMTP: Simple Mail Transfer Protocol (SMTP) is a standard protocol for **sending** emails from one mail server to another.

The most popular protocol for **retrieving** email on a mail server is the Post Office Protocol (POP) and the Internet Mail Access Protocol (IMAP).

POP is a standard mail protocol to receive and download emails from a remote mail server over a TCP/IP and client. Once emails are downloaded, they are deleted from the server. POP is a very simple protocol, and it lacks many features. For example, it does not support sorting or search. The latest version of POP, the version POP3, is still widely used. POP3 assumes that clients live in the same place. This can work if you bring it for a small company or a large enterprise.

IMAP is also a standard mail protocol for receiving emails from a mail server. When you read an email, you are connected to an external mail server, and data is transferred to your local device. IMAP only downloads a message when you click it, and emails are not deleted from the remote server. IMAP has many features, including folders, flags, and tags. IMAP is a good fit for a large company with individual email accounts. It works well even when the connection is slow because only the small header information is downloaded at first.

HTTPS is not technically a protocol, but it can be used to access email servers, specifically for web based email. For example, it's common for Microsoft Outlook to talk to a mobile device over HTTPS, or a custom mail protocol called ActiveSync [2].

Domain name service (DNS)

A DNS server is used to look up the mail exchanger record (MX record) for the recipient's domain. If you ran DNS lookup for gmail.com from the command line, the output of the MX records is shown below.

```
$ dig gmail.com MX
;; ANSWER SECTION:
gmail.com.      1800    IN      MX      5 gmail-smtp-in.l.google.com.
gmail.com.      1800    IN      MX      10 alt1.gmail-smtp-in.l.google.com.
gmail.com.      1800    IN      MX      40 alt4.gmail-smtp-in.l.google.com.
gmail.com.      1800    IN      MX      30 alt3.gmail-smtp-in.l.google.com.
gmail.com.      1800    IN      MX      20 alt2.gmail-smtp-in.l.google.com.
```

Figure 2:SMTP, IMAP, POP

The activity numbers indicate preferences, where the mail server with a lower priority number is more preferred. In Figure 2, email sent to gmail.com is sent to the SMTP mail server listening at port 25 of one of the gmail-smtp-in.l.google.com hosts. If the primary SMTP server is down, the sender's SMTP server will attempt to connect the next SMTP server in the list, which is alt1.gmail-smtp-in.l.google.com. If the second SMTP server is down, the sender's SMTP server will attempt to connect the next SMTP server, and so on. If no SMTP servers are available, the sender's SMTP server will keep the message in its queue and try the mail server again at a later time.

Attachment

An email attachment is sent along with an email message, commonly with Base64 encoding [3]. There is usually a size limit for an email attachment. For example, Outlook and Gmail limit the size of attachments to 25 MB (megabytes) as of June 2021. The number limit for attachments is 250 for Outlook and 500 for Gmail. Multipurpose Internet Mail Extension (MIME) [4] is a specification that allows the attachment to be sent over the Internet.

Traditional mail servers

Before we dive into distributed mail servers, let's say a few words through the history and learn why traditional mail servers work, as doing so will provide us with good context. Back in the day, most companies maintained their own mail servers. Sendmail and Exchange are two popular mail servers that are commonly used on a single server.

Traditional mail server architecture

Figure 3 describes what happens when one person sends an email to another, using traditional email servers.

Figure 3: Traditional mail servers

The process consists of 3 steps:

1. Alice logs in to her Outlook client, composes an email, and presses the "send" button. The client is sent to the Outlook mail server. The communication protocol between the Outlook client and the mail server is SMTP.
2. The Outlook mail server looks up the recipient's domain name (yahoo.com in this example) in the recipient's SMTP server. In this case, it is smtp.fr.yahoo.com. Next, it contacts the email to the Yahoo mail server. The communication protocol between the mail servers is also SMTP.
3. The Yahoo mail server stores the email until Bob retrieves it. Bob, the recipient

Once the email is stored in the Yahoo mail server, Bob can log into his Yahoo mail server to see the email. The communication protocol between Bob and the mail server is out of the scope of the discussion. There are many email client options for Bob to retrieve his emails from a remote mail server. He can use a web browser, or he can use a mobile app, or he can use a traditional email client such as Microsoft Outlook. The communication protocol between Bob's email client and the mail server is usually IMAP or HTTP.

Email functionality has come a long way since it was invented in the 1960s. from text-based format to rich features such as multimedia, threading, tags, labels, and more. But email protocols SMTP, POP3, and IMAP3 were invented a long time ago and they were not designed to support large scale systems as we now. Many extensions were made to the original protocols.

Distributed mail servers

Distributed mail servers are designed to address modern use cases and solve the problems of scale and resiliency. This section covers storage, metadata caching, high availability and scalability, and eventual consistency.

Email APIs

Email APIs can ease very different things for different mail clients, or at different stages of an email's life cycle. For example,

- SMTP/POP/IMAP APIs to name mailbox creation.
- SMTP communication between sender and receiver mail servers.
- Search API to index and search through a large number of email attachments.

Due to its simple interface and flexibility, we cover only some of the most important APIs for external. A common way for external to communicate is through the HTTP protocol.

Endpoints: POST /v1/messages

Send a message to the recipients in the To, Cc, and Bcc headers.

Endpoints: GET /v1/messages

Returns all folders of an email account.

Response:

```
HTTP/1.1 200 OK
Content-Type: application/ json
Content-Length: 1234
{ "data": [{ "id": "8627sgdsx23gj", "object": "folder", "name": "Inbox", "parent_id": null, "child_count": 0, "unread_count": 0, "system_folder": true }, { "id": "3124gdxcvbn76", "object": "folder", "name": "Phishing", "parent_id": null, "child_count": 0, "unread_count": 0, "system_folder": true }, ... ], "has_more": false }
```

Endpoints: GET /v1/messages/{ message_id}

Returns all messages under a folder. Keep in mind this is a highly simplified API. In reality, this needs to support pagination.

Response:

```
HTTP/1.1 200 OK
Content-Type: application/ json
Content-Length: 1234
{ "data": [{ "id": "1231fvsgabg567", "object": "message", "from": "johnsmith@example.com", "to": [{ "name": "", "email": "janedoe@example.com" }], "cc": [], "bcc": [], "subject": "Hello from the other side", "body": "I must call upon the author of Curious George.", "folder_id": "8627sgdsx23gj", "sent_at": null, "received_at": null, "is_read": false, "is_outgoing": false, "has_attachment": false, "is_phishing": false, "is_spam": false, "is_deleted": false, "created_at": null, "updated_at": null }, ... ], "has_more": true }
```

Distributed mail server architecture

Writing it is easy, but as we all know, email that handles a small number of users, is quite different from that handling tens of millions of users or more. For example, a single server can't handle such a large volume of traffic, so we need multiple servers. Storage may be large enough for a single server, but it's not a good idea to store all data on a single server, because it's not resilient. Accessing all data may be slow when it's located on a single server, and it's not scalable if we keep on adding more data to a single server. Finally, a single server can handle many email clients, but email delivery is not instantaneous if the recipient's mail server is down. We need to queue up data for a distributed email server to handle such cases.

Figure 4:High level design

Let us take a closer look at each component:

Mail server: Mail servers are responsible for receiving and processing emails.

Web servers: Web servers are public facing request/response servers, used to manage features such as login, signup, user profile, etc. In our design, email client support, such as sending and receiving emails, fetching emails, marking emails as read, etc. are handled by web servers.

Real-time servers: Real-time servers are responsible for pushing new email updates to clients in real-time. Real-time servers are stateful servers because they need to maintain persistent connections. To support real-time communication we have a few options, such as long polling and WebSocket. We discuss the pros and cons of each option in the design deep dive section. In our design, we use WebSocket because it provides a bidirectional communication channel possible and is quite scaling as a different.

Web is an example of a real-world mail server (Apache James [15]) that implements the IETF's Mail Store Application Protocol (JMAP) specification on Netbeans [16].

Metadata database: The database stores mail metadata including mail subject lines, from user, to users, etc. We discuss the database schema in the design deep dive section.

Attachment store: We choose object storage such as Amazon Simple Storage Service (S3) as the attachment store. S3 is a scalable object storage infrastructure that's suitable for storing large files such as images, videos, files, etc. Attachments can take up to 25MB in size. Netflix course from Amazon Web Services brought such a so good for this learning the material.

- Extra Google Cloudsets supports 5GB data type and for Netbeans Filestorage size for a limit is 2GB, the practical limit is less than 2GB in size.
- Another problem with storing attachments in Cassandra is that it can't put a row size so attachments take too much memory space.

Metadata cache: Since the most recent emails are repeatedly loaded for a client, caching recent emails in memory significantly improves the retrieval speed. We choose Redis or Memcached for our cache.

Search store: The search store is a distributed document store. It uses a data structure called inverted index [17] that supports very fast full-text searches. We will discuss in the deep dive section how it works.

Note that we have discussed some of the most important components to build distributed mail servers, let's assemble together two main workflows.

- Email sending flow.
- Email receiving flow.

Email sending flow

The email sending flow is shown in Figure 5.

Figure 5:Email sending flow

1. A user writes an email or selected and presses the "send" button. The request is sent to the load balancer.
2. The load balancer is used to route HTTP requests to the web and stores traffic to web servers.
3. Web servers are responsible for:
   a. Fetching sessions from session processing which is checked against user defined rules such as email size limit.
   b. Checking if the sender of the recipient's email address is valid. If the address is in the wrong form, the web server ensures the email data is spam and virus free. If so, email data is inserted into the sender's "Sent Folder" and recipient's "Inbox Folder". The email data is persisted in the metadata database and the attachment store.
   c. Message queue.
   d. SMTP outgoing workers consume the email data is passed in the sending queue. If the attachment is too large to fit in the queue, only a pointer to the attachment in the storage is passed. The pointer object references the attachment.
   e. SMTP outgoing workers get messages from the outgoing queue and make sure emails are spam and virus free.
   f. The outgoing email is stored in the "Sent Folder" of the storage layer.
   g. SMTP sending workers send the email to the recipient mail server.

Email message in the outgoing queue consists of the following: recipient, email subject, email body, and attachment. It is wise to keep small messages in memory for fast access. Most of the outgoing email messages such as notifications, reminders, and confirmations are created in the backend servers and sent through a message queue for further processing, such as spam and virus checks, and finally delivery.

We mention the use of the outgoing queue very clearly. If there are many emails stuck in the queue, we need to analyze the cause of the issue. Here are some possibilities:

- The recipient's mail server is unavailable. In this case, we need to retry sending the email at a later time. Exponential backoff [18] is a commonly used retry policy.
- Mail storage server is near full. In this case, we may need more computers to reduce the processing time.

Email receiving flow

The following diagram demonstrates the email receiving flow.

Figure 6:Email receiving flow

1. Incoming emails arrive at the SMTP mail servers.
2. The mail listener distributes traffic among SMTP servers. Email acceptance policy can be configured and applied at the SMTP processing servers. For example, we can reject emails from blacklisted domains or email addresses.
3. If the attachment size of email is too large to put into the queue, we put it into the attachment store [19].
4. Emails are put in the incoming email queue. The queue decouples the message from SMTP servers so they can be created quickly when the system is busy or when many slow email handlers are needed.
5. Mail processing workers are in charge of pulling emails off the incoming queue. For each email, retrieving simple rules, the following steps execute an email processed the validation.
   a. Virus and spam checks. Email can be marked as spam and virus data.
   b. Email is persisted in the storage layer. The email is ready to be read by users.
6. Real-time servers are Web/Socket servers that allow clients to receive new emails in real time.
7. For offline users, the modified data is stored in the storage layer. When user comes back online, the optimal client connects to web servers via AJAX/Ajax.
8. Web servers pull new emails from the storage layer and return them to the client.

Step 3 - Design Deep Dive

Now that we have looked at the parts of the email server, let's go deeper into some key components and examine how to make the system scalable.

- Metadata database
- Search
- Scalability
- Availability

Metadata database

In this section, we discuss the characteristics of email metadata, choosing the right database, data models and optimization lifecycle details.

Characteristics of email metadata

- Email headers are usually small and frequently accessed.
- Email bodies are large in size and rarely get retrieved by accessed. You normally only read an email once.
- Most of the read operations, such as listing folds, marking an email as read, and searching are isolated to an individual user. In other words, most queries only need to access data of a single user. Data access patterns are quite different from other systems.
- Data recency matters this design, more recently until recent days accessed emails. 50% of email queries are for data younger than 16 days [16].
- Data ownership rarely requirements. Data loss is not acceptable.

Choosing the right database

An email or Gmail-like, like Netbeans system is usually custom made to reduce inputoutput operations per second (IOPS) [20], as this can easily become a major constraint in the system. Choosing the right database is not easy. It is helpful to consider all the options we have and understand the pros and cons of each option.

- Relational database. The main advantage of using a relational database is flexibility. We are ready between to email lists, attachments, JSON, indexes, search queries out fast. However, relational databases are typically optimized for small chunks of data and random access patterns. Email data is huge in volume and access patterns are sequential by nature. Indexing can help, but it might cause more IOPS than the RDB data type is designed for. Moreover, relational database scaling and consumer/producer DB data type are not efficient. So relational database such as MySQL or Postgresqlis are not good fits.
- Distributed object storage system is also a viable option. It can handle large volumes of data and allows for random and fast retrieval. Some of the popular options that support strong consistency are Amazon S3, Netbeans Filestorage, Google Cloud Storage, and Azure Blob Storage. Many of these storage systems are quite mature and offer strong consistency guarantees. However, in an interview setting, we don't have time to design a distributed storage system. It's important to consider the following characteristics when we discuss object stores:
  - It simple options can be a single single of IMS.
  - Email metadata is immutable.
  - Arrangement is various data set.
  - It should be highly available and fault tolerant.
  - It should be easy to handle metadata changes.

Data model

One way to store data is to use one table (e.g as a partition key) as data for one user is stored on a single shard. One limitation of this data model is that messages are not shared among multiple users. Since this is a distributed system, it is not this interesting. It is something we should keep in mind.

Next let us discuss the tables. Cassandra may contains two partitions, the primary and the secondary.

- Partition key: responsible for distributing data across nodes. As a general rule, we want to spread the data evenly.
- Clustering key: responsible for sorting data within partition.

At a high level, an email service needs to support the following queries on the data layer:

- The first query is to get all folders.
- The second query is to display all emails for a specific folder.
- The third query is to display email content.
- The fourth query is to mark an email as read/ unread.
- Bonus point: get conversation threads.

Let's take a look at these one by one.

Query 1: get all folders for a user

As shown in Table 1, user ID is the partition key, as folders belonged to the same user are located in one partition.

K: Partition Key	
C1: Clustering Key (second-ing)	
C2: Clustering Key (descend-ing)	
folders_by_user	
user_id	UUID K
folder_id	UUID
folder_name	TEXT

Table 1:folders by user

Query 2: display all emails for a specific folder.

When a user loads their inbox, emails are usually sorted by timestamp, showing the most recent at the top. In order to store all emails for one folder in one partition, composite primary key is used. It holds all its unique identifier and the second key is used to sort. In this type 2 TABLEUNIT [21], and the is clustering key used to sort data in descending order.

emails_by_folder	
user_id	UUID K
folder_id	UUID K
email_id	TIMEUUID C1
from	TEXT
subject	TEXT
preview	TEXT
is_read	BOOLEAN

Table 2:Emails by folder

Query 3: Consolidated email content

In a Gmail-like, we are usually have to get detailed information about an email. The two tables, folder and email, are designed to support this query. The simple query looks like this:

```
SELECT * FROM emails_by_user WHERE email_id = 123;
```

An email can have multiple attachments, and these can be retrieved by the combination of email id and attachments fields.

emails_by_user	
user_id	UUID K
email_id	TIMEUUID C1
from	TEXT
subject	TEXT
preview	TEXT
body	TEXT
attachments	LIST<attachment>
attachments	
email_id	TIMEUUID C1
attachment_id	UUID C2
name	TEXT

Table 3:Emails by user

Query 4: Search all read or unread emails.

If our domain model was for a relational database, the query to find all read emails would look like this:

```
SELECT * FROM emails_by_user
WHERE user_id = 123 AND folder_id = 456AND is_read = true
ORDER BY email_id;
```

The query is fetch all unread emails would look very similar. We just need to change the query to `SELECT * FROM emails_by_user WHERE user_id = 123 AND folder_id = 456AND is_read = false ORDER BY email_id;`

Our data model, however, is designed for NoSQL. A NoSQL database normally only supports queries on partition and cluster keys. Since `is_read` is neither a partition nor a cluster key, we need to create a new table to support this query.

emails_by_folder_read_status	
user_id	UUID K
folder_id	UUID K
email_id	TIMEUUID C1
from	TEXT
subject	TEXT
preview	TEXT
is_read	BOOLEAN

Table 4:Emails by folder and read status

a email arrival service, but at our storage scale this does not work well.

This problem is commonly solved with denormalization in NoSQL. To support the read/unread queries, we denormalize the emails_by_folder data into two tables as shown in Table 6.

- read_emails: it stores all emails that are in read status.
- unread_emails: it stores all emails that are in unread status.

To mark an UNREAD email as READ, the email is deleted from unread_emails and then inserted to read_emails.

To fetch all unread emails for a specific folder, we can run a query like this:

```
SELECT * FROM unread_emails
WHERE user_id = user_id= and folder_id = <folder_id>
ORDER BY email_id;
```

read_emails

user_id	UUID K
folder_id	UUID C1
email_id	TIMEUUID C1
from	TEXT
subject	TEXT
preview	TEXT

unread_emails

user_id	UUID K
folder_id	UUID C1
email_id	TIMEUUID C1
from	TEXT
subject	TEXT
preview	TEXT

Table 4 Read and unread emails

Denormalization as shown above is a common practice. It makes the application code more complicated and harder to maintain, but it improves the read performance of these queries at scale.

Bonus point: conversation threads

Threads are a feature supported by many email clients. It groups email replies with their original message [8]. This allows users to retrieve all emails associated with one conversation. Traditionally, a thread is implemented using algorithms such as JWZ algorithm [9]. We will not go into detail about the algorithm, but just explain the core idea below to understand how Google implements the threading.

```
{
  "messages": [
    {"subject": "Lunch", "id": "7540f603-438C-4912-3B57-9B2E3FC9D8fgaus1.com",
     "In-Reply-To": "<CA3EPLq7Mu_qCr2Ki2Ug=0Ta8_qCrbrbbfqaus1.com",
     "References": "[\"<7540f603-438C-4912-3B57-9B2E3FC9D8fgaus1.com",
    ...
    ]
}
```

Message-Id	**The value of a message-ID. It is generated by a client while sending a message.**
In-Reply-To	The parent Message-Id to which the message replies.
References	A list of message-IDs related to a thread.

Table 5 Email header

With these fields, an email client can reconstruct mail conversations from messages, if all messages in the reply chain are preserved.

Consistency trade-off

Distributed databases that rely on replication for high availability must make a fundamental trade-off between consistency and availability. Consistency is very important for email storage. A user expects that we won't lose a single piece of email. However, in the event of a failover, the mailbox isn't accessible by clients, so their sync/update operation is paused until failover ends. It trades availability in favor of consistency.

Email deliverability

It is easy to set up a mail server and start sending emails. The hard part is to get emails actually delivered to a user's inbox. If an email ends up in the spam folder, it means there is a very high chance a recipient won't read it. Email spam is a huge issue. According to research done by Statista [10], more than 50% of all emails are spam. If we set up a new mail server, most likely our emails will end up in the spam folder because a new email server has no reputation. There are a couple of factors to consider to improve email deliverability.

Dedicated IPs. It is recommended to have dedicated IP addresses for sending emails. Email providers are less likely to accept emails from new IP addresses that have no history.

Classify emails. Send different categories of emails from different IP addresses. For example, you may want to avoid sending marketing and important emails from the same servers because it might make ISPs mark all emails as promotional.

Email sender reputation. Warm up new email server IP addresses slowly to build a good reputation, so big providers such as ORDoes, Gmail, Yahoo Mail, etc. are less likely to put our emails in the spam folder. According to Amazon Simple Email Service [23], it takes about 2 to 6 weeks to warm up a new IP address.

Ban spammers quickly. Spammers should be banned quickly before they have a significant impact on the server's reputation.

Feedback processing. It's very important to set up feedback loops with ISPs so we can keep the complaint rate low and ban spam accounts quickly. If an email fails to deliver or a user complains, one of the following outcomes occurs:

- Hard bounce. This means an email is rejected by an ISP because the recipient's email address is invalid.
- Soft bounce. A soft bounce indicates an email failed to deliver due to temporary unavailability such as an ISP's being too busy.
- Complaint. This means a recipient clicks the "report spam" button.

Figure 6 shows the process of collecting and processing bounces/complaints. We use separate queues for soft bounces, hard bounces, and complaints so they can be managed separately.

Figure 6 Handle feedback loop

Email authentication. According to the 2018 data breach investigation report provided by Verizon, phishing and pretexting represent 93% of breaches [21]. Some of the common techniques to combat phishing are: Sender Policy Framework (SPF) [22], Domainkeys Identified Mail (DKIM) [23], and Domain-based Message Authentication, Reporting and Conformance (DMARC) [24].

Figure 9 shows an example header of a Gmail message. As you can see, the sender dinthd.z0.com is authenticated by SPF, DKIM, and DMARC.

Message-ID	<6717.247304368.2501605304187739035.0100784800@dinthd.z0.com>
Created at:	Sun, May 2, 2021 at 6:41 PM (Delivered after 17 seconds)
From:	CA Alerts <alerts@dinthd.z0.com> Using SyzMailer
To:	[redacted]@gmail.com>
Subject	Your CDR account statement is ready
SPF:	PASS with IP 52.229.200.166 Learn more
DKIM:	'PASS' with domain mfd.z0.com Learn more
DMARC:	'PASS' Learn more

Figure 9 An example of a Gmail header

You don't need to remember all these terms. The important thing to keep in mind is that getting emails to work as intended is hard. It requires not only domain knowledge, but good relationships with ISPs.

Search

Basic mail search refers to searching for emails that contain any of the entered keywords in the subject or body. More advanced features include filtering by "From", "Subject", "Unread", or other attributes. On one hand, whenever an email is sent, received, or deleted, we need to perform reindexing. On the other hand, a search query can be issued at any time when a user presses the "search" button. This means the search feature in email systems has a lot more writes than reads. By comparison with Google search, email search has quite different characteristics, as shown in Table 6.

	Scope	Sorting	Accuracy
Google search	The whole internet	Sort by relevance	Indexing generally takes time, so some items may not show in the search result immediately.
Email search	User's own email box	Sort by attributes such as time, has attachment, date, is unread, etc.	Indexing should be near real-time, and the result has to be accurate.

Table 6 Google search vs email search

To support search functionality, we compare two approaches: Elasticsearch and native search embedded in the database.

Option 1: Elasticsearch

The high-level design for email search using Elasticsearch is shown in Figure 10. Because queries are mostly performed on the user's own email server, we can group underlying documents to the same node using user_id as the partition key.

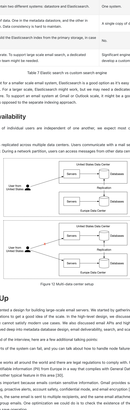

Figure 10 Elasticsearch

When a user clicks the "search" button, the user waits until the search response is received. A search request is synchronous. When events such as "email arrival", "receive email" or "delete email" are triggered, nothing related to search needs to be returned to the client. Reindexing is needed and it can be done with offline jobs. Kafka is used in the design to decouple services that trigger reindexing, from services that actually perform reindexing.

Elasticsearch is the most popular search engine database as of June 2021 [25] and it supports full-text search of emails very well. One challenge of adding Elasticsearch is to keep our primary email store in sync with it. One of the largest email providers in China, Tencent Qq Email, uses Elasticsearch [26].

Option 2: Custom search solution

Large-scale email providers usually develop their own custom search engines to meet their specific requirements. Designing an email search engine is a very complicated task and is out of the scope of this chapter. Here we only briefly touch on the disk I/O bottleneck, a primary challenge we still face for a custom search engine.

As shown in the back-of-the-envelope calculation, the size of the metadata and attachments added daily is at the petabyte (PB) level. Meanwhile, an email account can quickly have over half a million emails. The index bottleneck of the index server is usually disk I/O.

Since the process of building the index is write-heavy, a good strategy might be to use Log-Structured Merge-Tree (LSM) [27] to structure the index data on disk (Figure 11). The write path is optimized by only performing sequential writes. LSM trees are the core data structure behind databases such as BigTable, Cassandra, and RocksDB. When a new email arrives, it is first added to level 0 in-memory cache, and when data size in memory reaches the predefined threshold, data is merged to the next level. Another reason to use LSM is to separate data that change frequently from those that don't change. For example, email data usually doesn't change, but folder information tends to change more often due to different filter rules. In this case, we can separate them into two different sections, so that if a request is related to a folder change, we change only the folder and leave the email data alone.

If you are interested in reading more about email search, it is highly recommended you take a look at how search works in Microsoft Exchange servers [28].

Figure 11 LSM tree

Each approach has pros and cons:

Feature	Elasticsearch	Custom search engine
Scalability	Scalable to some extent.	Easier to scale as we can optimize the system for the email use case.
System complexity	Need to maintain two different systems: database and Elasticsearch.	One system.
Data consistency	Two copies of data. One in the metadata database, and the other in Elasticsearch. Data consistency is hard to maintain.	A single copy of data in the metadata database.
Data loss possible	No. Can rebuild the Elasticsearch index from the primary storage, in case of failure.	No.
Development effort	Easy to integrate. To support large scale email search, a dedicated Elasticsearch team might be needed.	Significant engineering effort is needed to develop a custom email search engine.

Table 7 Elastic search vs Custom search engine

A general rule of thumb is that for a smaller scale email system, Elasticsearch is a good option as it's easy to integrate and doesn't require significant engineering effort. For a larger scale, Elasticsearch might work, but we may need a dedicated team to develop and maintain the email search infrastructure. To support an email system of Gmail or Outlook scale, it might be a good idea to have a native search embedded in the database as opposed to the separate indexing cluster.

Scalability and availability

Since data access patterns of individual users are independent of one another, we expect most components in the system are horizontally scalable.

For better availability, data is replicated across multiple data centers. Users communicate with a mail server that is physically closer to them in the network topology. During a network partition, users can access messages from other data centers (Figure 12).

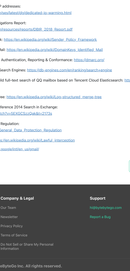

Figure 12 Multi-data center setup

Step 4 - Wrap Up

In this chapter, we have presented a design for building large-scale email servers. We started by gathering requirements and doing some back-of-the-envelope calculations to get a good idea of the scale. In the high-level design, we discussed how traditional email servers were designed and why they cannot satisfy modern use cases. We also discussed email API high-level designs for sending and receiving flows. Finally, we dived deep into metadata database design, email deliverability, search, and scalability.

If there's extra time at the end of the interview, here are a few additional talking points:

- Fault tolerance. Many parts of the system can fail, and you can talk about how to handle node failures, network issues, event delays, etc.
- Compliance. Email service works all around the world and there are legal regulations to comply with. For instance, we need to handle and store personally identifiable information (PII) from Europe in a way that complies with General Data Protection Regulation (GDPR) [29]. Legal intercept is another typical feature in this area [30].
- Security. Email security is important because emails contain sensitive information. Email provides safety features such as phishing protections, safe browsing, proactive alerts, account safety, confirmed mode, and email encryption [31].
- Optimizations. Sometimes, the sender and receiver are in multiple regions, and the same email attachment is stored several times in the object store (S3) in the group emails. One optimization we could do is to check the existence of the attachment in storage, before performing the expensive save operation.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Chapter summary

Reference Materials

[1] Number of Active Email Users https://financesonline.com/number-of-active-email-users/

[2] Outlook https://en.wikipedia.org/wiki/Outlook.com

[3] How Many Emails Are Sent Per Day in 2021? https://techjury.net/stats-about/how-many-emails-are-sent-per-day/

[4] RFC 1939 - Post Office Protocol - Version 3 http://www.faqs.org/rfcs/rfc1939.html

[5] ActiveSync https://en.wikipedia.org/wiki/ActiveSync

[6] Email attachment https://en.wikipedia.org/wiki/Email_attachment

[7] MIME https://en.wikipedia.org/wiki/MIME

[8] Threading https://en.wikipedia.org/wiki/Conversation_threading

[9] JWZ mail threading algorithm https://www.ietf.org/rfc/rfc5256.txt https://datatracker.ietf.org/doc/html/rfc8264

[10] Apache James https://james.apache.org/

[11] A JSON Meta Application Protocol (JMAP) Subprotocol for WebSocket https://datatracker.ietf.org/doc/rfc8887/

[12] Cassandra Limitations https://wiki.apache.org/cassandra/CassandraLimitations

[13] Inverted index https://en.wikipedia.org/wiki/Inverted_index

[14] Exponential backoff https://en.wikipedia.org/wiki/Exponential_backoff

[15] CQ Email System Optimization (in Chinese) https://www.slideshare.net/grossyu/cq06-slide-5433793

[16] IDFS https://en.wikipedia.org/wiki/IDFS

[17] UUID and timeuuid types https://docs.datastax.com/ja/cql-oss/3.3-beta/cql/cql_reference/cql_data_types_c.html

[18] Message threading https://www.jwz.org/doc/threading.html

[19] Global spam volume https://www.statista.com/statistics/420391/spam-email-traffic-share/

[20] Warming up dedicated IP addresses https://docs.aws.amazon.com/pinpoint/latest/developerguide/ip-warming.html

[21] 2018 Data Breach Investigations Report https://enterprise.verizon.com/resources/reports/dbir/2018/ 2018_Report.pdf

[22] Sender Policy Framework https://en.wikipedia.org/wiki/Sender_Policy_Framework

[23] DomainKeys Identified Mail https://en.wikipedia.org/wiki/DomainKeys_Identified_Mail

[24] Domain-based Message Authentication, Reporting & Conformance https://dmarc.org/

[25] DB-Engines Ranking of Search Engines https://db-engines.com/en/ranking/search+engine

[26] Refactoring practice: Tencent Qq Email's storage system in WCDB based on Tencent Cloud Elasticsearch https://www.programmersought.com/article/20097567327/

[27] Log-structured merge-tree https://en.wikipedia.org/wiki/Log-structured_merge-tree

[28] Microsoft Exchange Conference 2014 Search in Exchange https://www.youtube.com/watch?v=5KDCDcxbbb0&t=2729s

[29] General Data Protection Regulation https://en.wikipedia.org/wiki/General_Data_Protection_Regulation

[30] Lawful intercepion https://en.wikipedia.org/wiki/Lawful_interception

[31] Email safety https://safety.yahoo.net/intro_upload/

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Advertisers
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Gre Stories
Find Jobs
Prepare for Coding Interviews

Copyright ©2021-2026 BytebyteGo Inc. All rights reserved.